Can AI-LLM Write Your SQL? Yes. Should You Trust It? Let's Find Out.

- **Course:** OMIS 105 — Introduction to Database Management Systems
- **Quarter:** Fall 2026
- **Author:** Dr. Mahmoud Parsian
- **Last updated:** September 11, 2026

Table of Contents

Part I — When AI Gets It Wrong

1 — The Question Every Student Asks

> "Why do I need to learn SQL? I can just ask AI to write it for me."

Fair question. AI tools like ChatGPT and Claude **can** generate SQL. The queries they produce often look correct, run without errors, and return a table of results. So what's the problem?

The problem is that "runs without errors" and "gives the right answer" are two completely different things.

A query can execute perfectly and still return the **wrong number**. The wrong revenue. The wrong customer count. The wrong average. The wrong ranking. And if you don't know SQL, you will never notice.

In this document, we will use a realistic retail database to show you **seven real examples** where AI-generated SQL runs successfully but produces an incorrect answer — the kind of mistake that could cost a company money, damage a report, or lead to a bad business decision.

We will also show you **four examples** where AI gets it right — and explain why **you still need SQL knowledge** to verify and trust the output.

> **Note:** You don't need to understand every SQL keyword in this document yet. Focus on the **results** — what the AI got wrong and why it matters. You will learn all of these techniques during the course.

2 — The Company: BrightCart Retail

BrightCart is a small online retailer that sells office supplies and electronics. They have been in business since 2023 and track everything in a database: customers, products, orders, line items, and employees.

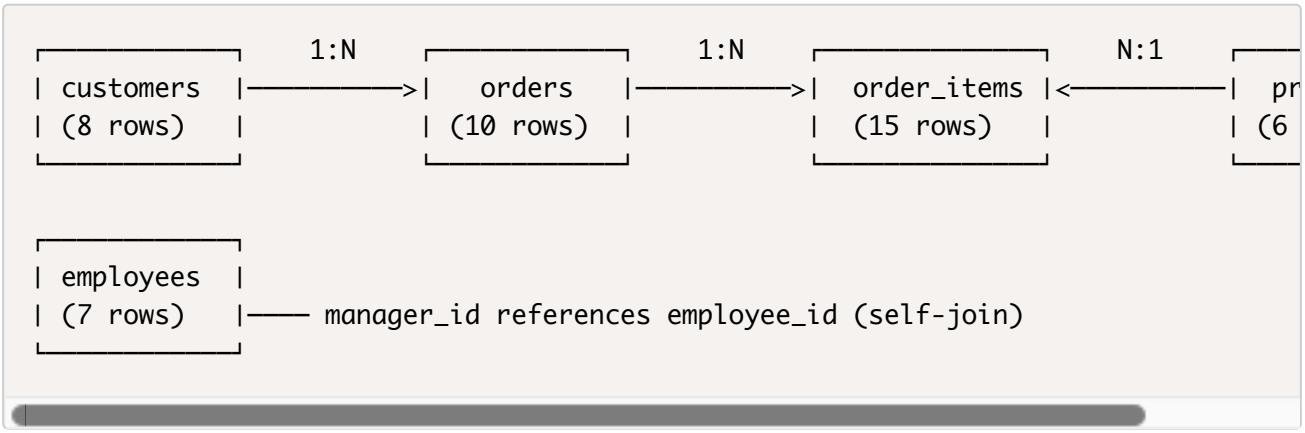
You are a new business analyst at BrightCart. Your manager gives you questions about the business, and you decide to let AI write the SQL. Let's see how that goes.

3 — The Database Schema

BrightCart's database has five tables. Here is what each one stores:

Table	What It Stores	Key Columns
`customers`	People who have accounts	name, city, state, join date
`products`	Items for sale	name, category, current price
`orders`	Purchase transactions	customer, date, status (Delivered / Cancelled / Returned / Pending)
`order_items`	Individual line items within an order	product, quantity, price at time of sale
`employees`	Company staff	name, department, salary, manager

How the Tables Connect



Foreign Key	In Table	Column	References	Meaning
Customer → Orders	`orders`	`customer_id`	`customers.customer_id`	Each order belongs to one customer; one customer can have many orders
				Each line

Order → Line Items	`order_items`	`order_id`	`orders.order_id`	item belongs to one order; one order can have many line items
Product → Line Items	`order_items`	`product_id`	`products.product_id`	Each line item refers to one product; one product can appear in many line items
Manager → Employee	`employees`	`manager_id`	`employees.employee_id`	Each employee may report to one manager, who is also an employee

The `order_items` table sits at the center — it connects orders to products and stores the actual quantity and price for each purchase. This is a common pattern in retail databases.

Two important things to notice:

1. **`products.unit_price`** is the **current** catalog price. But prices change over time. The price a customer actually paid is stored in **`order_items.sale_price`**. These two numbers can be different.
2. **`orders.status`** tells you whether an order was actually fulfilled. Not every order in the database represents real revenue — some were cancelled, returned, or are still pending.

4 — Setting Up the Data in qStudio

Open qStudio, connect to DuckDB, and run the following SQL blocks one at a time. This creates the five tables and fills them with realistic data.

4.1 — Create the Tables

```
CREATE OR REPLACE TABLE customers (
    customer_id   INTEGER PRIMARY KEY,
    first_name    VARCHAR,
    last_name     VARCHAR,
    city          VARCHAR,
    state         VARCHAR,
    join_date     DATE
);

CREATE OR REPLACE TABLE products (
    product_id    INTEGER PRIMARY KEY,
    product_name  VARCHAR,
    category      VARCHAR,
    unit_price    DECIMAL(10,2),
    discontinued  BOOLEAN
);

CREATE OR REPLACE TABLE orders (
    order_id      INTEGER PRIMARY KEY,
    customer_id   INTEGER REFERENCES customers(customer_id),
    order_date    DATE,
    status        VARCHAR
);

CREATE OR REPLACE TABLE order_items (
    item_id       INTEGER PRIMARY KEY,
    order_id      INTEGER REFERENCES orders(order_id),
    product_id    INTEGER REFERENCES products(product_id),
    quantity      INTEGER,
    sale_price    DECIMAL(10,2)
);

CREATE OR REPLACE TABLE employees (
    employee_id   INTEGER PRIMARY KEY,
    first_name    VARCHAR,
    last_name     VARCHAR,
    department    VARCHAR,
    hire_date     DATE,
    salary        DECIMAL(10,2),
    manager_id    INTEGER REFERENCES employees(employee_id)
);
```

4.2 — Insert the Data

```sql
-- 8 customers (two of them have never placed an order)
INSERT INTO customers VALUES
    (1, 'Alice', 'Chen',     'San Jose',      'CA', '2023-01-15'),
    (2, 'Bob',   'Martinez', 'San Francisco', 'CA', '2023-03-22'),
    (3, 'Carol', 'Johnson',  'Seattle',       'WA', '2023-06-10'),
    (4, 'David', 'Kim',      'Portland',      'OR', '2023-09-01'),
    (5, 'Emma',  'Williams', 'San Jose',      'CA', '2024-01-05'),
    (6, 'Frank', 'Brown',    'Denver',        'CO', '2024-02-14'),
    (7, 'Grace', 'Lee',      'Austin',        'TX', '2024-04-20'),
    (8, 'Henry', 'Davis',    'Phoenix',       'AZ', '2024-06-30');
```

```sql
-- 6 products (the Mouse and Desk Mat have had price increases)
-- Mouse was $24.99, now $29.99
-- Desk Mat was $39.99, now $45.99
INSERT INTO products VALUES
    (101, 'Wireless Mouse',    'Electronics', 29.99, false),
    (102, 'USB-C Hub',         'Electronics', 49.99, false),
    (103, 'Notebook 3-Pack',   'Office',      12.99, false),
    (104, 'Standing Desk Mat', 'Furniture',   45.99, false),
    (105, 'Webcam HD',         'Electronics', 79.99, false),
    (106, 'Paper Shredder',    'Office',      89.99, true);
```

```sql
-- 10 orders (not all were successfully delivered)
INSERT INTO orders VALUES
    (1001, 1, '2024-10-05', 'Delivered'),
    (1002, 1, '2024-11-12', 'Delivered'),
    (1003, 2, '2024-10-18', 'Delivered'),
    (1004, 2, '2024-11-25', 'Cancelled'),
    (1005, 3, '2024-10-02', 'Delivered'),
    (1006, 3, '2024-12-01', 'Returned'),
    (1007, 4, '2024-10-20', 'Delivered'),
    (1008, 5, '2024-11-01', 'Delivered'),
    (1009, 5, '2024-12-10', 'Pending'),
    (1010, 6, '2024-10-10', 'Delivered');
```

```sql
-- 15 line items (sale_price = what customer actually paid)
INSERT INTO order_items VALUES
    (1,  1001, 101, 2, 24.99),
    (2,  1001, 103, 1, 12.99),
    (3,  1002, 102, 1, 49.99),
    (4,  1003, 105, 1, 79.99),
    (5,  1003, 101, 1, 24.99),
    (6,  1004, 104, 2, 39.99),
    (7,  1005, 102, 1, 49.99),
    (8,  1005, 103, 3, 12.99),
    (9,  1006, 105, 1, 79.99),
    (10, 1007, 101, 3, 24.99),
    (11, 1007, 104, 1, 39.99),
    (12, 1008, 102, 2, 49.99),
    (13, 1008, 103, 1, 12.99),
    (14, 1009, 105, 1, 79.99),
    (15, 1010, 101, 1, 24.99);
```

```sql
-- 7 employees

-- ----------------
-- 2 managers first
-- ----------------
INSERT INTO employees VALUES
    (1, 'Sarah',  'Wilson',   'Sales',       '2020-03-15', 75000, NULL),
    (4, 'James',  'Thomas',   'Engineering', '2022-01-20', 95000, NULL);

-- ---------------------------------
-- 5 others: rest of employees
-- 2 have no department assigned yet
-- ---------------------------------
INSERT INTO employees VALUES
    (2, 'Mike',   'Taylor',   'Sales',       '2021-06-01', 62000, 1),
    (3, 'Lisa',   'Anderson', 'Marketing',   '2021-09-10', 68000, 1),
    (5, 'Nina',   'Patel',    'Engineering', '2022-08-15', 85000, 4),
    (6, 'Carlos', 'Rivera',    NULL,         '2024-07-01', 55000, 1),
    (7, 'Amy',    'Zhang',     NULL,         '2024-07-15', 58000, 4);
```

4.3 — Verify the Data

Run this quick check to confirm everything loaded:

```
SELECT 'customers'   AS table_name, COUNT(*) AS rows FROM customers
UNION ALL
SELECT 'products',    COUNT(*) FROM products
UNION ALL
SELECT 'orders',      COUNT(*) FROM orders
UNION ALL
SELECT 'order_items', COUNT(*) FROM order_items
UNION ALL
SELECT 'employees',   COUNT(*) FROM employees;
```

You should see:

```
┌──────────────┬────────┐
| table_name   | rows   |
|   varchar    | int64  |
├──────────────┼────────┤
| customers    |      8 |
| products     |      6 |
| orders       |     10 |
| order_items  |     15 |
| employees    |      7 |
└──────────────┴────────┘
```

5 — Example 1: The Missing Customers

The Business Question

Your manager asks: **“How many orders has each customer placed? I want to see all customers, including those who haven’t ordered yet.”**

What You Asked AI

You open a chatbot and type:

> “Using DuckDB, write SQL to show how many orders each customer has placed. I want to see all customers, including those with no orders. Tables: customers and orders, linked by customer_id.”

The prompt is clear — it even says “including those with no orders.” Let’s see what the AI produces.

What AI Generated

```sql
SELECT   c.first_name,
         c.last_name,
         COUNT(o.order_id) AS total_orders
FROM     customers c
         JOIN orders o
           ON c.customer_id = o.customer_id
GROUP BY c.first_name, c.last_name
ORDER BY total_orders DESC;
```

The AI's Answer

```
┌────────────┬────────────┬──────────────┐
| first_name | last_name | total_orders |
|  varchar   |  varchar  |    int64     |
├────────────┼────────────┼──────────────┤
| Alice      | Chen      |            2 |
| Carol      | Johnson   |            2 |
| Bob        | Martinez  |            2 |
| Emma       | Williams  |            2 |
| David      | Kim       |            1 |
| Frank      | Brown     |            1 |
└────────────┴────────────┴──────────────┘
```

The query runs perfectly. No errors. Six customers, each with at least one order. Looks good, right?

What's Wrong

BrightCart has **8** customers, not 6. Where are Grace Lee and Henry Davis?

The AI used a **JOIN** (also called an INNER JOIN), which only returns customers that have a **matching** row in the orders table. Grace and Henry have never placed an order, so they have no match — and they silently disappear from the results.

The manager explicitly asked for **all customers, including those who haven't ordered**. The AI ignored that requirement.

The Correct SQL

```
SELECT   c.first_name,
         c.last_name,
         COUNT(o.order_id) AS total_orders
FROM     customers c
         LEFT JOIN orders o
           ON c.customer_id = o.customer_id
GROUP BY c.first_name, c.last_name
ORDER BY total_orders DESC;
```

The Correct Answer

```
┌────────────┬────────────┬────────────────┐
| first_name | last_name | total_orders |
|  varchar   |  varchar  |    int64     |
├────────────┼────────────┼────────────────┤
| Alice      | Chen      |            2 |
| Emma       | Williams  |            2 |
| Carol      | Johnson   |            2 |
| Bob        | Martinez  |            2 |
| David      | Kim       |            1 |
| Frank      | Brown     |            1 |
| Grace      | Lee       |            0 |
| Henry      | Davis     |            0 |
└────────────┴────────────┴────────────────┘
```

Now we see all 8 customers. Grace and Henry show 0 orders — exactly the information the manager needed.

The Business Impact

If you used the AI's answer, you would report: *"Our least active customer has 1 order. Everyone is buying."* In reality, 25% of your customers have **never purchased anything**. That's critical information for a re-engagement campaign. The AI's answer hid it completely.

> **The Lesson:** `JOIN` and `LEFT JOIN` are not the same thing. A single wrong word can make entire rows of data disappear from your results — and the query will still run without any error.

---

6 — Example 2: The Wrong Revenue

The Business Question

Your manager asks: **"What was our total revenue from all orders?"**

What You Asked AI

> "Write a DuckDB query to calculate total revenue from all orders. Tables: customers, products, orders, order_items."

Notice what's missing from the prompt: you didn't specify **which price column** to use — because you don't know there are two. The AI has to guess, and it guesses wrong.

What AI Generated

```
SELECT SUM(oi.quantity * p.unit_price) AS total_revenue
FROM   order_items oi
       JOIN products p
         ON oi.product_id = p.product_id;
```

The AI's Answer

total_revenue
852.78

The query runs. One clean number. You report $852.78 to your manager.

What's Wrong

The AI calculated revenue using `products.unit_price` — the **current** catalog price. But two products have had price increases since these orders were placed:

Product	Price When Sold	Current Price
Wireless Mouse	$24.99	$29.99
Standing Desk Mat	$39.99	$45.99

The price customers **actually paid** is stored in `order_items.sale_price`. The AI joined to the wrong table and used the wrong price column.

The Correct SQL

```
SELECT SUM(quantity * sale_price) AS total_revenue
FROM   order_items;
```

The Correct Answer

total_revenue
799.78

The real revenue is **$799.78**, not $852.78. The AI's answer was ** 53.00 too high** — a 6.6% overstatement.

The Business Impact

A `6.6%` revenue overstatement in a report might seem small on `10` orders. On a company doing `$10 million` in annual sales, the same mistake overstates revenue by `$660,000`. That's the kind of error that leads to wrong financial forecasts, incorrect tax filings, and very difficult conversations with auditors.

> **The Lesson:** Databases often store the same type of information (like "price") in multiple places. The **current** price and the **historical** price are different numbers. AI doesn't understand your business well enough to know which one to use. You do — but only if you understand the schema and the SQL.

---

7 — Example 3: The Inflated Numbers

The Business Question

Your manager asks: **"How many orders did we successfully deliver in Q4 2024?"**

What You Asked AI

> "How many orders were delivered in Q4 2024? Table: orders."

You said "delivered," but you didn't tell the AI that the table has a `status` column that needs to be filtered. The AI treats "delivered" as a time-range description, not as a SQL filter condition.

What AI Generated

```sql
SELECT COUNT(*) AS orders_delivered
FROM   orders
WHERE  order_date BETWEEN '2024-10-01' AND '2024-12-31';
```

The AI's Answer

orders_delivered
10

Ten orders delivered in Q4. You put “10” in the quarterly report.

What’s Wrong

The AI counted **all** orders in that date range, regardless of status. But not all orders were delivered:

order_id	status	Should Count?
1001	Delivered	Yes
1002	Delivered	Yes
1003	Delivered	Yes
1004	**Cancelled**	**No**
1005	Delivered	Yes
1006	**Returned**	**No**
1007	Delivered	Yes
1008	Delivered	Yes
1009	**Pending**	**No**
1010	Delivered	Yes

Three orders were not successfully delivered: one was cancelled, one was returned, and one is still pending.

The Correct SQL

```sql
SELECT COUNT(*) AS orders_delivered
FROM   orders
WHERE  order_date BETWEEN '2024-10-01' AND '2024-12-31'
  AND  status = 'Delivered';
```

The Correct Answer

orders_delivered
7

The real answer is **7**, not 10. The AI inflated the count by **43%**.

The Business Impact

If you report 10 deliveries when only 7 actually shipped, your fulfillment metrics look artificially strong. The operations team thinks everything is fine. Meanwhile, 30% of orders are failing — a serious problem that nobody is investigating because the report said 10.

> **The Lesson:** AI doesn't understand your business rules. It doesn't know that "delivered" means `status = 'Delivered'`, not just "exists in the orders table." Business logic lives in the data, and only someone who understands both SQL and the domain can write the correct filter.

---

8 — Example 4: The NULL Trap

The Business Question

Your manager asks: **"List all employees who are NOT in the Sales department."**

What AI Generated

```sql
SELECT first_name,
       last_name,
       department
FROM   employees
WHERE  department != 'Sales';
```

The AI's Answer

first_name	last_name	department
Lisa	Anderson	Marketing
James	Thomas	Engineering
Nina	Patel	Engineering

Three employees. The rest must all be in Sales, right?

What's Wrong

BrightCart has **7** employees. Two are in Sales (Sarah and Mike). That leaves **5** who are not in Sales. But the AI only found 3.

Where are Carlos Rivera and Amy Zhang? They are recent hires whose department has not been assigned yet — their `department` column is **NULL** (empty/unknown).

In SQL, `NULL` is special. It does not equal anything, and it does not *not-equal* anything either. The comparison `NULL != 'Sales'` does not return TRUE — it returns **NULL**, which SQL treats as "unknown," and the row is excluded.

This is one of the most common mistakes in SQL, and AI makes it frequently.

The Correct SQL

```sql
SELECT first_name,
       last_name,
       department
FROM   employees
WHERE  department != 'Sales'
   OR  department IS NULL;
```

The Correct Answer

first_name	last_name	department
Lisa	Anderson	Marketing
James	Thomas	Engineering
Nina	Patel	Engineering
Carlos	Rivera	*NULL*
Amy	Zhang	*NULL*

Now we see all 5 employees who are not in Sales, including the two unassigned new hires.

The Business Impact

If you used the AI's answer to send a company-wide announcement to "everyone outside Sales," Carlos and Amy would never receive it. If you used it to calculate headcount for budget planning, your non-Sales departments would appear to have 3 people instead of 5 — understaffed by 40%.

> **The Lesson:** NULL is not a value — it means "unknown." Standard comparisons (`=`, `!=`, `>`, `<`) do not work with NULL. You must use `IS NULL` or `IS NOT NULL`. AI frequently forgets this rule, and the query runs without any error or warning.

---

9 — Example 5: The Wrong Average

The Business Question

Your manager asks: **"What is our average order value for delivered orders?"**

What You Asked AI

> "Calculate the average order value for delivered orders. Tables: orders and order*items, linked by order*id."

"Average order value" sounds clear in English, but it is ambiguous in SQL: do you mean the average of each **line item's value**, or the average of each **order's total**? The AI picks the simpler interpretation — and gets the wrong number.

What AI Generated

```sql
SELECT ROUND(AVG(quantity * sale_price), 2) AS avg_order_value
FROM   order_items oi
       JOIN orders o
         ON oi.order_id = o.order_id
WHERE  o.status = 'Delivered';
```

The AI's Answer

avg_order_value
46.65

$46.65 per order. You put this number in the financial summary.

What's Wrong

The AI averaged the **individual line items**, not the **order totals**.

Here are the delivered orders and their actual totals:

order_id	Line Items	Order Total
1001	$(2 \times 24.99) + (1 \times 12.99)$	$62.97
1002	(1 × $49.99)	$49.99
1003	$(1 \times 79.99) + (1 \times 24.99)$	$104.98
1005	$(1 \times 49.99) + (3 \times 12.99)$	$88.96
1007	$(3 \times 24.99) + (1 \times 39.99)$	$114.96
1008	$(2 \times 49.99) + (1 \times 12.99)$	$112.97
1010	(1 × $24.99)	$24.99

The average of these 7 order totals is: $(62.97 + 49.99 + 104.98 + 88.96 + 114.96 + 112.97 + 24.99) \div 7 =$ **79.97**

The AI's answer of `$46.65` is **42% too low** because it averaged 12 line items instead of 7 order totals. An order with 3 line items got 3 times the weight of an order with 1 line item.

The Correct SQL

```sql
SELECT ROUND(AVG(order_total), 2) AS avg_order_value
FROM (
    SELECT   o.order_id,
             SUM(oi.quantity * oi.sale_price) AS order_total
    FROM     orders o
             JOIN order_items oi
               ON o.order_id = oi.order_id
    WHERE    o.status = 'Delivered'
    GROUP BY o.order_id
) AS order_totals;
```

The Correct Answer

avg_order_value
79.97

The real average order value is **$79.97**, not $46.65.

The Business Impact

Average order value is a key metric for pricing strategy, marketing spend, and revenue forecasting. If you report `$47` instead of `$80`, management might conclude that customers aren't spending enough and launch unnecessary discounts — cutting into margins to solve a problem that doesn't exist.

> **The Lesson:** "Average" depends on **what you're averaging**. Are you averaging line items or orders? AI often gets the "grain" wrong — it computes the average at the wrong level of detail. The query runs fine. The number looks reasonable. But it's the wrong number.

---

10 — Example 6: The Broken Ranking — AI Ignores Ties

The Business Question

Your manager asks: **"Rank our products by total units sold from delivered orders. If two products sold the same quantity, they must share the same rank."**

What AI Generated

```sql
SELECT   p.product_name,
         SUM(oi.quantity) AS units_sold,
         ROW_NUMBER() OVER (ORDER BY SUM(oi.quantity) DESC) AS sales_rank
FROM     order_items oi
         JOIN products p
           ON oi.product_id = p.product_id
         JOIN orders o
           ON oi.order_id = o.order_id
WHERE    o.status = 'Delivered'
GROUP BY p.product_name;
```

The AI's Answer

product_name	units_sold	sales_rank
Wireless Mouse	7	1
Notebook 3-Pack	5	2
USB-C Hub	4	3
Standing Desk Mat	1	4
Webcam HD	1	5

Five products, each with a unique rank. Looks clean, right?

What's Wrong

Look at the bottom two products: the Standing Desk Mat and the Webcam HD **both sold exactly 1 unit**. They are tied. The manager explicitly asked for tied products to share the same rank.

But the AI used `ROW_NUMBER()`, which assigns a unique sequential number to every row — it **never produces ties**. The Desk Mat got rank 4 and the Webcam got rank 5 purely based on arbitrary internal ordering, not on actual performance.

The correct function is `RANK()`, which gives tied rows the same rank number.

The Correct SQL

```sql
SELECT   p.product_name,
         SUM(oi.quantity) AS units_sold,
         RANK() OVER (ORDER BY SUM(oi.quantity) DESC) AS sales_rank
FROM     order_items oi
         JOIN products p
           ON oi.product_id = p.product_id
         JOIN orders o
           ON oi.order_id = o.order_id
WHERE    o.status = 'Delivered'
GROUP BY p.product_name;
```

The Correct Answer

product_name	units_sold	sales_rank
Wireless Mouse	7	1
Notebook 3-Pack	5	2
USB-C Hub	4	3
Standing Desk Mat	1	**4**
Webcam HD	1	**4**

Now both products that sold 1 unit share **rank 4** — exactly what the manager asked for.

The Business Impact

Imagine the ranking is used to decide which products to discontinue: *"Cut the lowest-ranked product."* With the AI's answer, only the Webcam (rank 5) gets flagged. The Desk Mat (rank 4) looks like it performed better — even though they had identical sales. You might discontinue one product while

keeping another that performed exactly the same. Fair decisions require accurate rankings.

> **The Lesson:** `ROW_NUMBER()`, `RANK()`, and `DENSE_RANK()` are three different functions that look similar but behave differently with ties. AI frequently picks the wrong one. If your business question involves ties, you need to know the difference — and verify which function the AI chose.

---

11 — Example 7: The LIMIT Illusion — AI Can't Do "Per Customer"

The Business Question

Your manager asks: **"For each customer who has placed an order, show me their single most recent order with the order date and status."**

What AI Generated

```
SELECT   c.first_name,
         c.last_name,
         o.order_id,
         o.order_date,
         o.status
FROM     customers c
         JOIN orders o
           ON c.customer_id = o.customer_id
ORDER BY o.order_date DESC
LIMIT    6;
```

The AI's Answer

first_name	last_name	order_id	order_date	status
Emma	Williams	1009	2024-12-10	Pending
Carol	Johnson	1006	2024-12-01	Returned
Bob	Martinez	1004	2024-11-25	Cancelled
Alice	Chen	1002	2024-11-12	Delivered
Emma	Williams	1008	2024-11-01	Delivered
David	Kim	1007	2024-10-20	Delivered

Six rows. Six customers have orders, so one row per customer, right?

What's Wrong

Look carefully: **Emma Williams appears twice** (orders 1009 and 1008). And **Frank Brown is completely missing** — his most recent order (1010, October 10) didn't make the global top-6 cutoff.

The AI used `ORDER BY ... LIMIT 6`, which returns the **6 most recent orders across all customers**. It does not return the most recent order **for each** customer. `LIMIT` is a global cap — it has no concept of "per group."

To get one row per customer, you need a **window function** that partitions the data by customer and picks the top row within each partition.

The Correct SQL

```sql
SELECT first_name,
       last_name,
       order_id,
       order_date,
       status
FROM (
    SELECT c.first_name,
           c.last_name,
           o.order_id,
           o.order_date,
           o.status,
           ROW_NUMBER() OVER (
               PARTITION BY c.customer_id
               ORDER BY o.order_date DESC
           ) AS rn
    FROM   customers c
           JOIN orders o
             ON c.customer_id = o.customer_id
) AS ranked
WHERE  rn = 1
ORDER BY order_date DESC;
```

The Correct Answer

first_name	last_name	order_id	order_date	status
Emma	Williams	1009	2024-12-10	Pending
Carol	Johnson	1006	2024-12-01	Returned
Bob	Martinez	1004	2024-11-25	Cancelled
Alice	Chen	1002	2024-11-12	Delivered
David	Kim	1007	2024-10-20	Delivered
Frank	Brown	1010	2024-10-10	Delivered

Now each customer appears **exactly once** with their most recent order. Frank is back.

The Business Impact

If you used the AI's answer to contact each customer about their most recent experience, Emma would receive **two messages** while Frank would receive **none**. You would also miss that Frank's last order was back in October — he may be at risk of churning and could benefit from a follow-up. Meanwhile, the duplicated effort on Emma wastes resources and looks unprofessional.

> **The Lesson:** `LIMIT` restricts the total number of rows returned — it cannot select "the top N per group." Whenever you need the top row (or top N rows) **within each group**, you need a window function like `ROW_NUMBER() OVER (PARTITION BY ...)`. AI frequently reaches for `LIMIT` when a window function is required.

Part II — When AI Gets It Right

The examples above might make it seem like AI always fails. It doesn't. AI **can** produce correct SQL — and when it does, it saves time. But here's the catch: **you can only benefit from AI's correct answers if you have the knowledge to recognize them as correct.**

12 — Example 8: Top 3 Customers — Can You Verify It?

The Business Question

Your manager asks: **"Who are our top 3 customers by total spending on delivered orders?"**

What AI Generated

```sql
SELECT   c.first_name,
         c.last_name,
         SUM(oi.quantity * oi.sale_price) AS total_spent
FROM     customers c
         JOIN orders o
           ON c.customer_id = o.customer_id
         JOIN order_items oi
           ON o.order_id = oi.order_id
WHERE    o.status = 'Delivered'
GROUP BY c.first_name, c.last_name
ORDER BY total_spent DESC
LIMIT    3;
```

The AI's Answer

first_name	last_name	total_spent
David	Kim	114.96
Emma	Williams	112.97
Alice	Chen	112.96

Is This Correct?

Yes. This query is correct. It joins the right tables, filters by delivered status, uses `sale_price` (not `unit_price`), groups by customer, sorts descending, and limits to the top 3.

So Why Do You Still Need to Know SQL?

Because **how would you know it's correct if you didn't understand the SQL?**

To verify this answer, you need to understand:

Concept	Why You Need It
`JOIN` (three tables)	Are customers, orders, and items connected correctly?
`WHERE status = 'Delivered'`	Did the AI exclude non-delivered orders? (Example 3 showed it doesn't always do this)
`SUM(oi.quantity * oi.sale_price)`	Did it use the right price column? (Example 2 showed it sometimes picks the wrong one)
`GROUP BY`	Is it grouping by customer, not by order or by line item? (Example 5 showed the grain matters)
`ORDER BY ... DESC`	Is it sorting highest-to-lowest?
`LIMIT 3`	Is it showing the top 3, not the bottom 3?

If you cannot read the SQL, you cannot answer any of these questions. You are forced to **blindly trust** that the AI got it right — even though the seven examples above proved it often doesn't.

> **The Lesson:** Even when AI writes correct SQL, your job is to **verify** it. Verification requires understanding. Understanding requires learning. There is no shortcut.

---

13 — Example 9: Revenue by Category — Can You Prove It?

The Business Question

Your manager asks: **"Show me total revenue by product category for delivered orders only. Include the number of orders and total units sold for each category. Use the actual price the customer paid."**

What AI Generated

```sql
SELECT   p.category,
         COUNT(DISTINCT o.order_id) AS num_orders,
         SUM(oi.quantity)           AS total_units,
         SUM(oi.quantity * oi.sale_price) AS total_revenue
FROM     order_items oi
         JOIN orders o
           ON oi.order_id = o.order_id
         JOIN products p
           ON oi.product_id = p.product_id
WHERE    o.status = 'Delivered'
GROUP BY p.category
ORDER BY total_revenue DESC;
```

The AI's Answer

category	num_orders	total_units	total_revenue
Electronics	7	12	454.88
Office	3	5	64.95
Furniture	1	1	39.99

Is This Correct?

Yes. This query is correct. But don't take our word for it — **prove it yourself.**

Student Challenge

> **Your task:** This query is correct, but can you explain **why**? Examine the SQL carefully and identify **at least five things** the AI got right. For each one, reference which earlier example showed AI getting that same thing wrong.

Here is a checklist to guide your analysis:

What to Check	Which Example Showed This Going Wrong?
Does the JOIN type matter here? Could `LEFT JOIN` change the result?	Example 1 (The Missing Customers)
Which price column does the query use — `sale_price` or `unit_price`?	Example 2 (The Wrong Revenue)
Does the query filter by order status?	Example 3 (The Inflated Numbers)
Could NULL values in any column affect the result?	Example 4 (The NULL Trap)
Is the `GROUP BY` at the right level of detail? Is `COUNT(DISTINCT ...)` necessary?	Example 5 (The Wrong Average)
Does the query need `RANK()` or `ROW_NUMBER()`? Why or why not?	Example 6 (The Broken Ranking)
Does `LIMIT` appear? Should it?	Example 7 (The LIMIT Illusion)

Bonus Verification

If you want to double-check the numbers, you can verify the Electronics total manually. The delivered orders containing electronics products are:

order_id	product	qty	sale_price	line_total
1001	Wireless Mouse	2	$24.99	$49.98
1002	USB-C Hub	1	$49.99	$49.99
1003	Webcam HD	1	$79.99	$79.99
1003	Wireless Mouse	1	$24.99	$24.99
1005	USB-C Hub	1	$49.99	$49.99
1007	Wireless Mouse	3	$24.99	$74.97
1008	USB-C Hub	2	$49.99	$99.98
1010	Wireless Mouse	1	$24.99	$24.99

Electronics total:

`$49.98` + `$49.99` + `$79.99` + `$24.99` + `$49.99` + `$74.97` + `$99.98` + `$24.99`

= **$454.88** ✓

All three categories combined:

`$454.88` + `$64.95` + `$39.99` = **$559.82**

— which matches the total delivered revenue from our earlier examples.

> **The Lesson:** The best way to trust AI output is to **understand it deeply enough to verify it yourself**. When you can read the SQL, check the logic, and confirm the numbers, AI becomes a powerful accelerator. Without that understanding, it's just a black box.

---

14 — Example 10: Department Salary Report — AI Remembers NULL

The Business Question

Your manager asks: **"For each department that has employees assigned, show the headcount, average salary, lowest salary, and highest salary."**

What AI Generated

```sql
SELECT   department,
         COUNT(*)              AS num_employees,
         ROUND(AVG(salary), 0) AS avg_salary,
         MIN(salary)           AS min_salary,
         MAX(salary)           AS max_salary
FROM     employees
WHERE    department IS NOT NULL
GROUP BY department
ORDER BY avg_salary DESC;
```

The AI's Answer

department	num_employees	avg_salary	min_salary	max_salary
Engineering	2	90000	85000	95000
Sales	2	68500	62000	75000
Marketing	1	68000	68000	68000

Is This Correct?

Yes. Let's verify against the raw data:

`employee`	`department`	`salary`
Sarah Wilson	Sales	$75,000
Mike Taylor	Sales	$62,000
Lisa Anderson	Marketing	$68,000
James Thomas	Engineering	$95,000
Nina Patel	Engineering	$85,000
Carlos Rivera	*NULL*	$55,000
Amy Zhang	*NULL*	$58,000

Engineering: 2 employees, avg $(95,000 + 85,000) \div 2$ = $90,000 ✓

Sales: 2 employees, avg $(75,000 + 62{,}000) \div 2$ = $68,500 ✓

Marketing: 1 employee, avg $68,000 ✓

Carlos and Amy are correctly excluded by `WHERE department IS NOT NULL`.

Why This Matters

Remember Example 4? There the AI **forgot** about NULL and silently dropped employees from the result. Here, the AI handled NULL correctly — it used `IS NOT NULL` to exclude unassigned employees, which is exactly what the manager requested.

But you could only confirm this by reading the SQL. If you didn't understand the `WHERE department IS NOT NULL` clause, you might not realize that two employees were intentionally excluded — or worse, you might not notice they were missing at all.

Think About It

> **Question for you:** What would happen if we **removed** the `WHERE department IS NOT NULL` line? Would the query still run? What would appear in the `department` column for Carlos and Amy? Would the averages change?
>
> Try it in qStudio and see for yourself.
>
> **The Lesson:** When AI gets NULL handling right, it's because the pattern happened to match its

> training data — not because it understood your business rules. Your job is to confirm that the NULL handling matches what the manager actually wanted.

15 — Example 11: Full Product Catalog with Sales — AI Uses a Subquery

The Business Question

Your manager asks: **"Show me every product in our catalog — including products that have never been sold — along with their total units sold and total revenue from delivered orders. Products with no sales should show zero."**

This is a challenging query because it combines two separate ideas: "all products" (even unsold ones) and "only delivered revenue."

What AI Generated

```
SELECT   p.product_name,
         p.category,
         p.unit_price              AS current_price,
         COALESCE(d.total_units, 0)   AS total_units_sold,
         COALESCE(d.total_revenue, 0) AS total_revenue
FROM     products p
         LEFT JOIN (
             SELECT   oi.product_id,
                      SUM(oi.quantity)                AS total_units,
                      SUM(oi.quantity * oi.sale_price) AS total_revenue
             FROM     order_items oi
                      JOIN orders o
                        ON oi.order_id = o.order_id
             WHERE    o.status = 'Delivered'
             GROUP BY oi.product_id
         ) d ON p.product_id = d.product_id
ORDER BY total_revenue DESC;
```

The AI's Answer

product_name	category	current_price	total_units_sold	total_revenue
USB-C Hub	Electronics	49.99	4	199.96
Wireless Mouse	Electronics	29.99	7	174.93
Webcam HD	Electronics	79.99	1	79.99
Notebook 3-Pack	Office	12.99	5	64.95
Standing Desk Mat	Furniture	45.99	1	39.99
Paper Shredder	Office	89.99	0	0.00

Is This Correct?

Yes. This is a sophisticated query, and the AI nailed it. Let's break down **why** it works:

Technique	Why It's Here	Which Lesson Applies
`LEFT JOIN` to the subquery	Ensures **all** products appear, even the Paper Shredder with zero sales	Example 1 (Missing Customers)
Subquery uses `sale_price`	Revenue is based on what customers paid, not today's catalog price	Example 2 (Wrong Revenue)
Subquery filters `status = 'Delivered'`	Only counts revenue from fulfilled orders	Example 3 (Inflated Numbers)
`COALESCE(d.total_units, 0)`	Converts NULL (from the LEFT JOIN) to 0 for unsold products	Example 4 (NULL Trap)
`GROUP BY oi.product_id` in subquery	Aggregates at the product level before joining back	Example 5 (Wrong Average)

Verification

The six product totals add up:

`$199.96` + `$174.93` + `$79.99` + `$64.95` + `$39.99` + `$0.00` = **`$559.82`**

— which matches the total delivered revenue we verified in earlier examples. ✓

Notice that the Paper Shredder (product 106, discontinued) is included with zero sales. It was never ordered, so the `LEFT JOIN` produces `NULL` values, and `COALESCE` converts them to `0`. This is exactly what the manager asked for.

Think About It

> **Question for you:** The `current_price` column shows today's catalog price (`$29.99` for the Wireless Mouse), but the `total_revenue` column is based on the historical `sale_price` (`$24.99` per unit). That means `$29.99 × 7 units = $209.93`, but the actual revenue is `$174.93`. Why is this difference a feature, not a bug? What would go wrong if the query used `current_price` to calculate revenue?
>
> *(Hint: Review Example 2.)*
>
> **The Lesson:** When AI produces a correct complex query, every piece of it — the JOIN type, the price column, the status filter, the NULL handling, the aggregation level — represents a decision that could have gone wrong. You saw seven examples where it did. The ability to read this query and confirm that **every decision was the right one** is exactly why you need to learn SQL.

Closing

16 — The Verdict

AI is a powerful tool. It can save time, suggest approaches you haven't considered, and help you write SQL faster — **once you already understand SQL**.

But AI is not a substitute for understanding. Here is what we demonstrated across eleven examples:

Example	AI's Answer	Correct Answer	Error
1. Customer count	6 customers	8 customers	**2 customers missing (25%)**
2. Total revenue	$852.78	$799.78	**$53.00 overstated (6.6%)**
3. Delivered orders	10 orders	7 orders	**3 extra counted (43% inflated)**
4. Non-Sales employees	3 employees	5 employees	**2 employees invisible (40%)**
5. Average order value	$46.65	$79.97	**$33.32 too low (42%)**
6. Product ranking	5 unique ranks	2 products share rank 4	**Tied products given different ranks**
7. Most recent order	Emma appears twice	Each customer once	**1 customer duplicated, 1 missing**

Every one of these queries ran without a single error message. Every one returned a clean, professional-looking table. **Every one was wrong.**

In a real company, these mistakes lead to:

- Wrong financial reports sent to executives
- Marketing campaigns that target the wrong customers
- Inventory decisions based on inflated demand
- Budget allocations based on missing headcount
- Strategic plans built on incorrect metrics
- Unfair product or employee evaluations
- Duplicated outreach and missed at-risk customers

You would not trust a financial report you cannot read. Do not trust a SQL query you cannot read either.

---

17 — Summary of Lessons

Part I — When AI Gets It Wrong

#	Trap	What AI Did Wrong	What You Need to Know
1	Wrong JOIN type	Used `JOIN` instead of `LEFT JOIN` — dropped customers with no orders	The difference between `JOIN` and `LEFT JOIN`
2	Wrong price column	Used the current catalog price instead of the historical sale price	How to read a schema and pick the right column
3	Missing business filter	Counted all orders, not just delivered ones	How to apply business rules with `WHERE`
4	NULL behavior	Used `!=` which silently skips NULL values	How NULL works in SQL comparisons
5	Wrong aggregation grain	Averaged line items instead of order totals	How `GROUP BY` and subqueries control the level of detail
6	Wrong ranking function	Used `ROW_NUMBER()` instead of `RANK()` — ignored ties	The difference between `ROW_NUMBER()`, `RANK()`, and `DENSE_RANK()`
7	LIMIT instead of window	Used `LIMIT` for a "per group" question — duplicated some, missed others	How `ROW_NUMBER() OVER (PARTITION BY ...)` solves per-group problems

Part II — When AI Gets It Right

#	What AI Got Right	Why You Still Need SQL
8	Top 3 customers — correct JOINs, filter, price column, grouping	You must verify each decision the AI made; blind trust is not an option
9	Revenue by category — correct across all dimensions	Systematic verification ties back to every lesson in Part I
10	Department salaries — correctly handled NULL exclusion	Confirming NULL logic requires understanding `IS NOT NULL`
11	Product catalog with sales — LEFT JOIN, subquery, COALESCE, status filter	Every technique from Part I appears in one query; you must verify them all

The Bottom Line

Learn SQL **first**. Then use AI to write it **faster**. Never the other way around.

---

18 — How to Prompt AI Better

This document proved that blind trust in AI-generated SQL is dangerous. But the answer is not to avoid AI — it is to **use it properly**. Once you understand SQL, AI becomes a powerful accelerator. Here are seven rules for getting better results.

Rule 1 — Include the Schema

Don't say "write a query for our orders database." Give the AI the actual `CREATE TABLE` statements. The more context it has about your columns, types, and relationships, the better its choices will be.

Weak prompt:

> "Write SQL to find total revenue by customer."

Strong prompt:

> "Given these tables: [paste your CREATE TABLE statements]. Write SQL to find total revenue per customer. Revenue should use `order_items.sale_price` (the price at time of sale), not `products.unit_price` (the current catalog price). Only include orders where `status = 'Delivered'`."

The strong prompt eliminates Examples 2 and 3 before the AI even starts.

Rule 2 — State the Business Rules Explicitly

AI does not know your business. If “revenue” only counts delivered orders, say so. If employees with NULL departments should be excluded, say so. If tied products must share the same rank, say so. Every unstated assumption is an opportunity for a mistake.

Think of it this way: the seven failures in Part I all came from business rules the AI had to **guess**. The fewer guesses, the fewer errors.

Rule 3 — Specify Edge Cases

Mention NULLs, ties, cancelled orders, customers with no purchases — whatever applies to your data. Examples 1, 4, 6, and 7 all involved edge cases that the AI ignored.

A useful prompt addition:

> “Note: some employees have NULL in the department column. Handle them according to [your requirement].”

Rule 4 — Name the SQL Dialect

Don’t just say “SQL” — say **“DuckDB SQL.”** Left unstated, the AI defaults to whatever dialect shows up most in its training data — usually MySQL or generic ANSI SQL — and that can produce queries that fail outright, or worse, run and quietly behave differently.

Weak prompt:

> “Write SQL to get the top 3 products per category.”

Strong prompt:

> “Write **DuckDB SQL** to get the top 3 products per category.”

A few real examples of what goes wrong without it:

MySQL-flavored answer	What happens in DuckDB
`` `column_name` `` (backticks)	Syntax error — DuckDB expects double quotes
`LIMIT 10, 3` (offset, count)	Syntax error — DuckDB wants `LIMIT 3 OFFSET 10`
`DATE_FORMAT(order_date, '%Y-%m')`	Function doesn’t exist — DuckDB uses `strftime(order_date, '%Y-%m')`

Naming the dialect also works in your favor, not just against errors: it unlocks DuckDB-specific features — `QUALIFY`, `PIVOT` / `UNPIVOT`, `LIST` / `UNNEST` — that you’ll meet later in this course and that can make a query shorter than a generic ANSI SQL answer would be.

Rule 5 — Ask the AI to Explain Its Choices

After the AI generates SQL, ask follow-up questions:

> "Why did you use JOIN instead of LEFT JOIN?"
>
> "Why did you use ROW_NUMBER instead of RANK?"
>
> "Which price column did you choose and why?"
>
> "How does this query handle NULL values?"

If the AI cannot justify its decisions clearly, that is a red flag. A correct query should have a clear explanation for every choice.

Rule 6 — Verify with Known Data

Run the AI's query on a small dataset where you already know the correct answer — like the BrightCart data in this document. If the numbers don't match your hand calculations, you have caught a bug before it reaches a real report.

This is the same verification technique we used in Examples 8–11. It works because you built the knowledge to check the output.

Rule 7 — Read the SQL Before You Run It

Even if you trust the AI, **read the query first.** Use this checklist:

Check	What to Look For	Which Example Showed This Failing
JOIN type	`JOIN` vs `LEFT JOIN` — will any rows be silently dropped?	Example 1
Price column	`sale_price` vs `unit_price` — historical or current?	Example 2
Status filter	Is `WHERE status = 'Delivered'` present when needed?	Example 3
NULL handling	Does the query use `IS NULL` / `IS NOT NULL` correctly?	Example 4
Aggregation level	Is `GROUP BY` at the right grain (orders vs line items)?	Example 5
Ranking function	`ROW_NUMBER()` vs `RANK()` vs `DENSE_RANK()` — does it handle ties?	Example 6
LIMIT vs window	Is `LIMIT` being used where `PARTITION BY` is needed?	Example 7

This checklist is exactly what you built by studying the eleven examples in this document. It turns you from a passive consumer of AI output into an active, critical reviewer.

Put It All Together: A Reusable Prompt Template

Rules 1–4 are about what you give the AI *before* it writes anything. Bake all four into one template, fill in the brackets, and you eliminate most of the mistakes from Part I before the AI writes a single line:

```
Given these tables (DuckDB SQL):
[paste your CREATE TABLE statements]

Write a DuckDB query to: [your specific question].

Business rules:
- [e.g., "revenue" means order_items.sale_price, not products.unit_price]
- [e.g., only include orders where status = 'Delivered']
- [e.g., include every customer, even those with zero orders]

Edge cases to handle:
- [e.g., some rows have NULL in [column] — include or exclude them?]
- [e.g., if two rows tie, they must share the same rank]
```

Rules 5–7 are about what you do *after* the AI answers: ask it to explain its choices, verify the numbers against data you already know, and read the query yourself before you trust it.

Try It Yourself

Go back to the weak prompt from Example 5 (Section 9): *"Calculate the average order value for delivered orders. Tables: orders and orderitems, linked by orderid."* That prompt produced the wrong answer — $46.65 instead of 79.97$.

Using the template above, rewrite it as a strong prompt. Then paste your version into an AI chatbot. Does it get $79.97 on the first try? If not, which rule did your prompt still miss?

The Real Skill

The goal of this course is not to make AI unnecessary — it is to make **you** the person who can tell whether AI's answer is right or wrong.

That is a skill no AI can replace.

---

19 — Flagship Example: One Bad Prompt, One Good Prompt, Two Very Different Answers

Every example so far showed you a wrong query and a hand-corrected one. This example does something closer to what you'll actually do at work: it shows the **same question**, asked two different ways — a weak prompt and a prompt that applies Rules 1–3 from Section 18 — and lets you watch the AI produce two completely different business answers from the exact same data.

The Business Question

Your manager asks: **"Which of our customers are repeat customers — meaning they have more than one delivered order? For each one, show how many delivered orders they placed and their total spending across those delivered orders."**

This is exactly the kind of question a marketing team asks to build a loyalty program: find the best, most loyal customers and reward them. It's a completely reasonable, everyday business request.

The Weak Prompt

You're in a hurry, so you type the question into a chatbot without applying any of the rules from Section 18:

> "Write SQL to find customers with more than one order, and their total spending. Tables: customers,

> orders, order_items."

Notice everything this leaves out: no schema, no mention of "delivered," no mention of which price column to use, and no warning that a single order can contain several line items.

What AI Generated

```sql
SELECT   c.first_name,
         c.last_name,
         COUNT(*) AS order_count,
         SUM(oi.quantity * p.unit_price) AS total_spent
FROM     customers c
         JOIN orders o
           ON c.customer_id = o.customer_id
         JOIN order_items oi
           ON o.order_id = oi.order_id
         JOIN products p
           ON oi.product_id = p.product_id
GROUP BY c.first_name, c.last_name
HAVING   COUNT(*) > 1
ORDER BY total_spent DESC;
```

The AI's Answer

first_name	last_name	order_count	total_spent
Bob	Martinez	3	201.96
Emma	Williams	3	192.96
Carol	Johnson	3	168.95
David	Kim	2	135.96
Alice	Chen	3	122.96

Five repeat customers, ready to load into the loyalty program. The query ran without a single error.

What's Wrong — Three Mistakes at Once

This one query repeats three mistakes from Part I simultaneously.

1. **`COUNT(*)` counts line items, not orders — the fan-out bug.** Joining `order_items` turns each order into one row per line item. David Kim placed exactly **one** order (order 1007), but that order has two line items — so `COUNT(*)` reports 2, and he is wrongly flagged as a repeat

customer. This is the same wrong-grain mistake as Example 5, now showing up as a headcount instead of an average.

2. **No status filter — the same mistake as Example 3.** Bob's second "order" (1004) was **Cancelled**. Emma's second "order" (1009) is still **Pending**. Carol's second "order" (1006) was **Returned**. None of these should count as evidence of repeat business or contribute revenue. The AI counted all of them anyway.

3. **`products.unit_price` instead of `order_items.sale_price` — the same mistake as Example 2.** Every dollar figure above uses today's catalog price, not what the customer actually paid.

Stack those three mistakes together and the damage compounds. Once you remove the cancelled, returned, and pending orders, **only Alice Chen has more than one delivered order.** The other four names on this list don't belong there at all — and this isn't a report headed to a drawer, it's a list of people about to receive a "thank you for your loyalty" email they didn't earn.

The Revised Prompt

Now apply Rules 1–3 from Section 18 — schema, business rules, edge cases — before sending anything to the AI:

> "Given these tables (DuckDB SQL): [paste the `customers`, `orders`, and `order_items` `CREATE TABLE` statements]. Write a DuckDB query to find repeat customers — customers with more than one **delivered** order. Use `COUNT(DISTINCT o.order_id)` to count orders, not `COUNT(*)`, because one order can have several line items. For revenue, use `order_items.sale_price` (the price the customer actually paid), not `products.unit_price`. Only count orders where `status = 'Delivered'` — exclude Cancelled, Returned, and Pending orders from both the order count and the total spending."

Every sentence in this prompt closes one of the three gaps above.

What AI Generated This Time

```sql
SELECT   c.first_name,
         c.last_name,
         COUNT(DISTINCT o.order_id) AS order_count,
         SUM(oi.quantity * oi.sale_price) AS total_spent
FROM     customers c
         JOIN orders o
           ON c.customer_id = o.customer_id
         JOIN order_items oi
           ON o.order_id = oi.order_id
WHERE    o.status = 'Delivered'
GROUP BY c.first_name, c.last_name
HAVING   COUNT(DISTINCT o.order_id) > 1
ORDER BY total_spent DESC;
```

The Correct Answer

`first_name`	`last_name`	`order_count`	`total_spent`
Alice	Chen	2	112.96

One repeat customer, not five. And 112.96 — *not* 122.96 — is her real total. (You already verified this exact number once before, in Example 8's Top 3 Customers list.)

Side-by-Side: The Two Prompts

	Weak Prompt	**Revised Prompt**
Schema included?	No	Yes (Rule 1)
"Delivered only" stated?	No	Yes (Rule 2)
Price column specified?	No	Yes (Rule 1)
Multi-line-item orders addressed?	No	Yes (Rule 3)
Repeat customers found	**5**	**1**
Top total spending shown	**$201.96**	**$112.96**
Correct?	No	Yes

Same manager, same question, same data — **the only thing that changed was the prompt** — and it changed the business decision entirely. Load the AI's first answer into a loyalty campaign and you'd email four customers (Bob, Carol, David, Emma) who don't qualify, while still correctly reaching the one who does.

> **The Lesson:** A weak prompt and a strong prompt are not two versions of the same question — on real data, they can produce two completely different business decisions. Everything in Section 18 exists to close that gap. Rules 1–3 alone — schema, business rules, edge cases — turned five wrong names into one right one.

References

1. Do You Still Need to Learn SQL in the Age of AI?

2. Why You Should Still Learn SQL — Even When AI Can Write It For You

3. Should you still learn SQL (in the age of LLMs)?

4. Is SQL Still Relevant in 2026? SQL in the Age of LLMs

5. How Anthropic enables self-service data analytics with Claude

6. Anthropic is telling you that Agentic Analytics is not just text-to-SQL
